COSC204
Computer Systems

Pointers to Routines

Overview

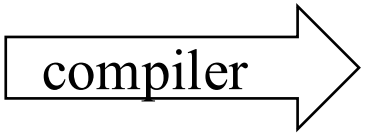
- Last Lecture
  - Structs and Unions
- This Lecture
  - Pointers to Routines
  - Source
    - K&R:
      - Brian Kernighan and Dennis Ritchie (1988), The C Programming Language. 2nd Edition. Prentice Hall. ISBN:0-13-110362-8
- Next Lecture
  - Operating Systems (with Zhiyi Huang)

Compiler

- The compiler converts a high level language description of a program into an assembly language description of a program

```
int another;

int main(void)
{
another = 0x1234;
return 0;
}
```

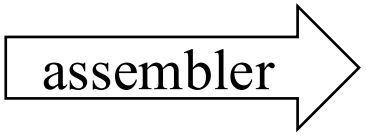



```
main:
    ldd #$1234
    std another+0,pcr
    rts
another:
    rmb 2
```

Assembler

- The assembler converts an assembly language description of a program into a machine code description of the program

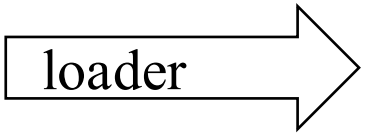
```
main:
   ldd #$1234                    4000  CC 12 34
   std another+0,pcr             4003  ED 8D 00 01
   rts                           4007  39
another:                         4008  00 00
   rmb 2
```



Loader

- The loader loads the program into memory
- This program is loaded at $4000

```
4000   CC 12 34
4003   ED 8D 00 01
4007   39
4008   00 00
```



$CC	$4000
$12	$4001
$34	$4002
$ED	$4003
$8D	$4004
$00	$4005
$01	$4006
$39	$4007
$00	$4008
$00	$4009

Memory

What is the address of:

another (this is a pointer to an integer)

main (this is a pointer to a routine)

```
int another;

int main(void)
{
another = 0x1234;
return 0;
}
```

compiler

```
main:
    ldd #$1234
    std another+0,pcr
    rts
another:
    rmb 2
```

assembler

```
4000  CC 12 34
4003  ED 8D 00 01
4007  39
4008  00 00
```

loader

$CC	$4000	main
$12	$4001	
$34	$4002	
$ED	$4003	
$8D	$4004	
$00	$4005	
$01	$4006	
$39	$4007	
$00	$4008	another
$00	$4009	

Pointers To Routines

- To declare a pointer to a routine:

```
int (*method)(int param);
```

- What kind of routine can `method` point to?

- To assign a pointer to a routine:

```
method = square;
```

- To call the routine

```
result = method(6);
```

- Why does this look like a function call?

Pointers to Routines

- What is the output?

```
int main(int argc, char *argv[])
{
int (*method)(int param);
int result1, result2;

method = identity;
result1 = method(6);

method = square;
result2 = method(6);

printf("Got:%d %d\n", result1, result2);

return 0;
}
```

```
int identity(int parameter)
{
return parameter;
}

int square(int parameter)
{
return parameter * parameter;
}
```

Typedefs

- Typedefs are often used to simplify declarations

```
typedef int (*operation)(int a, int b);

int plus(int a, int b)
{
return a + b;
}

int main(int argc, char *argv[])
{
operation op;

op = plus;
printf("%d\n", op(2, 3));
}
```

Pointers to Routines

- Pointers to routines can be use anywhere a variable can be used
- As members of structs:

```
typedef struct tree_t
{
struct tree_t *left, *right;
void  *data;
int (*operation)(void *data);
} tree;
```

- As parameters to other routines (use a typedef):

```
int method_x(int (*p)(int z), int x)
{
return p(x);
}

int square(int parameter)
{
return parameter * parameter;
}

int main(int argc, char *argv[])
{
int (*method)(int param);
int result1;

method = square;
result1 = method_x(method, 6);
printf("Got:%d\n", result1);

return 0;
}
```

Advantages

- If we put routine pointers into the `struct` then we gain a level of abstraction
- A queue could have a `print()` method, so too could a stack, a tree, and so on
- We call `ob->print()` without needing to know (exactly) which print method is called

Modules

- In C we break programs into functional *modules*
  - Modules are a little bit like *classes*
  - Each module *encapsulates* one behaviour
  - Modules give us a way of *data hiding*

- Add pointers to structs as parameters to methods in modules
  - We call this an *abstract data type* (ADT)
  - We communicate with an ADT via its public *interface*
    - So that the implementation details are hidden

- This is beginning to look a lot like Java

Modules

- C programs consist of a number of `.c` and `.h` files
- Put the ADT declaration in header (.h) files
- Put the implementation in source (.c) files

Queue

- From last lecture:

```
queue *queue_new(void)
void queue_enqueue(queue *this, uint32_t value)
uint64_t queue_dequeue(queue *this)
```

- And we should have a method to delete a queue:

```
void queue_delete(queue *this)
```

ADT Queue

```
typedef struct q_queue queue;

struct q_queue
{
queue_item *head;

queue *(*new)(void);
void (*print)(queue *this);
void (*enqueue)(queue *this, uint32_t value);
int64_t (*dequeue)(queue *this);
void (*delete)(queue *this);
};
```

Constructor

- The constructor “creates” the object, initialises it, and sets up the function pointers

```
queue *queue_new(void)
{
queue *this = malloc(sizeof(queue));
this->head = NULL;

this->new = queue_new;
this->print = queue_print;
this->enqueue = queue_enqueue;
this->dequeue = queue_dequeue;
this->delete = queue_delete;
return this;
}
```

Terminology

- What is this called:

```
struct q_queue
{
queue_item *head;

queue *(*new)(void);
void (*print)(queue *this);
void (*enqueue)(queue *this,
uint32_t value);
int64_t (*dequeue)(queue *this);
void (*delete)(queue *this);
};
```

- What is this called:

```
void queue_delete(queue *this)
{
/* HOMEWORK */
}
```

- What is this called:

```
queue *queue_new(void)
{
queue *this = malloc(sizeof(queue));
this->head = NULL;

this->new = queue_new;
this->print = queue_print;
this->enqueue = queue_enqueue;
this->dequeue = queue_dequeue;
this->delete = queue_delete;
return this;
}
```

queue.h

```
#pragma once

#include <stdint.h>

typedef struct q_item queue_item;
typedef struct q_queue queue;

struct q_queue
{
queue_item *head;

queue *(*new)(void);
void (*print)(queue *this);
void (*enqueue)(queue *this, uint32_t value);
int64_t (*dequeue)(queue *this);
void (*delete)(queue *this);
};

queue *queue_new(void);
```

queue.c

```
#include <stdlib.h>
#include "queue.h"

struct q_item
{
uint32_t value;
struct q_item *next;
};

void queue_print(queue *this)
{
if (this->head == NULL)
   printf("QUEUE EMPTY");
else
   {
   printf("Head");
   for (queue_item *current = this->head; current != NULL; current = current->next)
      printf("->%lu", current->value);
   }
puts("");
}
```

main.c

```
#include <stdio.h>
#include "queue.h"

int main(void)
{
queue *bus_stop;

bus_stop = queue_new();
bus_stop->print(bus_stop);
bus_stop->enqueue(bus_stop, 101);
bus_stop->print(bus_stop);
bus_stop->enqueue(bus_stop, 431);
bus_stop->print(bus_stop);
bus_stop->enqueue(bus_stop, 345);
bus_stop->print(bus_stop);

printf("%lld\n", bus_stop->dequeue(bus_stop));
printf("%lld\n", bus_stop->dequeue(bus_stop));
printf("%lld\n", bus_stop->dequeue(bus_stop));
printf("%lld\n", bus_stop->dequeue(bus_stop));
return 0;
}
```

RAII

- One of the important C++ “patterns” is RAII
  - Resource Allocation Is Initialization
- This pattern states that “a resource is a class invariant, and is tied to object lifetime” (Wikipedia)

- This means you should only ***allocate*** in the constructor and only ***deallocate*** in the destructor
  - Allocation might be calling `malloc()`, opening a file, etc.
  - Deallocation might be calling `free()`, closing a file, etc.

  - Of course, it’s sometimes necessary to break RAII
    - Such as a dynamic array (a C++ `std::vector<>`)

RAII

- If the constructor is successful
  - All resources have been allocated
  - The object is fully constructed
  - The object is ready for use
- If the destructor is successful
  - All resources have been deallocated
  - The object did not “leak” resources
    - All malloc() calls have been matched with free() calls
    - All file opens have been matched with file closes
- “Invariant” means a resource managed by the object does not change during the lifetime of the object
  - A file object does not close one file and open another

Summary

- Routines are just data at some memory location
  - Remember, we have a von Neumann architecture
- We can take the address of a routine
- We can put address into structures
- We can *abstract* behaviour in this way
  - We don't need to know which method is being called, only that the right method is being called

- Of course, the members of these structs can be changed at run-time!
  - Unlike C++ or Java which prevent this in classes

Homework

- Write:

```
void queue_delete(queue *this)
```

- If the queue is not empty then it should empty the queue first
  - Hint:
    - To empty the queue call `queue_dequeue()` until the result is negative